เอกสารการออกแบบ: Challenges & Solutions (10 คะแนน)

การพัฒนาระบบ Real-time Team Chat บน Task Board มีความท้าทายทางเทคนิคที่ต้องจัดการ เพื่อให้ระบบทำงานได้อย่างถูกต้องและรองรับผู้ใช้งานจำนวนมากได้ ซึ่งมีแนวทางแก้ไขดังนี้:

Challenge 1: Message Order (ลำดับข้อความ)

- **ปัญหา:** หากระบบเชื่อถือเวลา (Timestamp) ที่ส่งมาจากฝั่ง Client จะทำให้เกิดปัญหาลำดับข้อความผิดเพี้ยน เนื่องจากนาฬิกาของอุปกรณ์แต่ละเครื่องอาจไม่ตรงกัน (Clock Skew) หรือเกิดจากความหน่วงของอินเทอร์เน็ต (Latency) ที่ทำให้ข้อความที่พิมพ์ทีหลังเดินทางมาถึงเซิร์ฟเวอร์ก่อน
- **Solution:** ใช้การประทับเวลาที่ฝั่งเซิร์ฟเวอร์ (Server-side Timestamping) ร่วมกับการสร้าง Sequence Number
- **เหตุผล:** ให้ Chat Service ทำหน้าที่เป็น Single Source of Truth ทันทีที่เซิร์ฟเวอร์ได้รับข้อความ จะทำการแนบ Timestamp ของเซิร์ฟเวอร์เองลงไป พร้อมรันหมายเลข Sequence ประจำห้องแชท (boardId) นั้นๆ เมื่อ Client รับข้อความไปแสดงผล จะยึดการเรียงลำดับตาม Timestamp และ Sequence จากเซิร์ฟเวอร์เสมอ ทำให้ทุกคนในห้องเห็นลำดับข้อความตรงกันเป๊ะ

Challenge 2: Message Delivery Guarantee (การันตีการส่งข้อความ)

- **ปัญหา:** เครือข่ายอินเทอร์เน็ตอาจไม่เสถียร (โดยเฉพาะบนมือถือ) หากผู้ใช้กดส่งข้อความแล้วเน็ตหลุดชั่วคราว ผู้ใช้จะไม่รู้ว่าข้อความไปถึงเซิร์ฟเวอร์หรือยัง หากผู้ใช้กดส่งซ้ำรัวๆ อาจทำให้เกิดข้อความซ้ำซ้อน (Duplicate Messages) ในฐานข้อมูล
- **Solution:** ใช้กลไก Acknowledgement (ACK) ร่วมกับ Idempotency Key (รหัสระบุตัวตนของข้อความ)
- **เหตุผล:** เมื่อ Client ส่งข้อความ จะต้องสร้าง UUID (Idempotency Key) แนบไปด้วยเสมอ เมื่อ Chat Service บันทึกลง MongoDB สำเร็จ จะส่งสัญญาณ ACK กลับไปหา Client หาก Client ไม่ได้รับ ACK ภายใน 3 วินาที จะทำการส่งข้อความเดิม (พร้อม UUID เดิม) ไปใหม่ เซิร์ฟเวอร์จะเช็ค UUID นี้ก่อน ถ้าพบว่ามีในระบบแล้ว จะไม่บันทึกซ้ำ แต่จะส่ง ACK กลับไปให้เฉยๆ ป้องกันปัญหาข้อความเบิ้ลได้อย่างเด็ดขาด

Challenge 3: Scalability (การขยายระบบเมื่อมีผู้ใช้หลายพันคน)

- **ปัญหา:** Node.js ทำงานแบบ Single-thread แม้จะรับโหลดได้ดี แต่เมื่อถึงจุดหนึ่ง CPU/Memory ของ 1 เครื่องจะเต็ม เมื่อเราเพิ่มจำนวนเครื่อง (Instance 2, 3...) ไว้หลัง API Gateway จะเกิดปัญหาว่า User A ที่เชื่อมต่อกับเครื่องที่ 1 จะส่งข้อความหา User B ที่บังเอิญเชื่อมต่อกับเครื่องที่ 2 ไม่ได้ เพราะ WebSocket เป็นการเชื่อมต่อที่ผูกติดกับเครื่องนั้นๆ (Stateful)
- **Solution:** ทำ Horizontal Scaling โดยใช้ **Redis Pub/Sub** ทำหน้าที่เป็น Message Broker (Backplane) ตรงกลาง
- **เหตุผล:** เราให้ Chat Service ทุกเครื่องเชื่อมต่อกับ Redis เมื่อ User A ส่งข้อความเข้าเครื่องที่ 1 เครื่องที่ 1 จะเอาข้อความนี้ไป Publish ลงในช่องสัญญาณ (Channel) ของบอร์ดนั้นบน Redis ทันที เครื่องที่ 2 ที่ Subscribe ช่องสัญญาณนี้อยู่ ก็จะได้รับข้อความ และสามารถ Push ข้อความนั้นผ่าน WebSocket ของตัวเองไปให้ User B ได้ทันที ทำให้ระบบ Scale ได้อย่างไร้ขีดจำกัด

Challenge 4: Chat History (ประวัติการแชทเก็บอย่างไร? เก็บนานแค่ไหน?)

- **ปัญหา:** ข้อมูลแชทเป็นข้อมูลที่โตไวมาก (High Volume) หากเก็บข้อความทั้งหมดของทุกบอร์ดตั้งแต่เปิดระบบไว้ใน MongoDB อย่างเดียวตลอดไป ฐานข้อมูลจะบวม กินพื้นที่จัดเก็บราคาสูง และทำให้การ Query ดึงประวัติช้าลงอย่างมาก
- **Solution:** ใช้กลไก Data Tiering (แบ่งระดับชั้นการจัดเก็บข้อมูล) เป็น Hot Data และ Cold Data
- **เหตุผล:** * **Hot Data:** เก็บประวัติแชทย้อนหลังช่วง 3 - 6 เดือนล่าสุด ไว้ใน **MongoDB** เพื่อให้ระบบสามารถดึงข้อมูลมาแสดงผลตอนผู้ใช้ไถหน้าจอได้อย่างรวดเร็วที่สุด
    - **Cold Data:** ข้อความที่เก่ากว่า 6 เดือน ระบบจะรัน Background Job อัตโนมัติเพื่อนำข้อมูลไปบีบอัดเป็นไฟล์ (เช่น JSON หรือ Parquet) แล้วย้ายไปจัดเก็บใน **AWS S3** (Object Storage) ซึ่งมีราคาถูกกว่ามาก หากผู้ใช้ต้องการค้นหาแชทเก่ามากๆ ค่อยให้ระบบไปดึงจาก S3 มาแสดง วิธีนี้ช่วยรักษาประสิทธิภาพของ Database หลัก และประหยัดค่าใช้จ่ายได้มหาศาล